

Components in React



Functional Components

Stateless, simple UI functions



Class Components

Stateful components with lifecycle



Props

Pass data from parent to child



State

Component-specific reactive data



Hooks

Reusable logic for functional components



Context

Share data across the tree



Lifecycle

Manage component phases



Higher-Order Components

Enhance components by wrapping

FUNDAMENTALS

Introduction to Components

What is a Component?

A component is a reusable, independent piece of code that returns a UI element to be rendered on the screen. Components are the fundamental building blocks of React applications, functioning as JavaScript functions or classes that accept inputs (props) and return React elements describing what should appear on the screen.

Each component encapsulates its own structure, styling, and behaviour, making the code modular and maintainable. Components can represent anything from a simple button to a complex form or an entire page section.

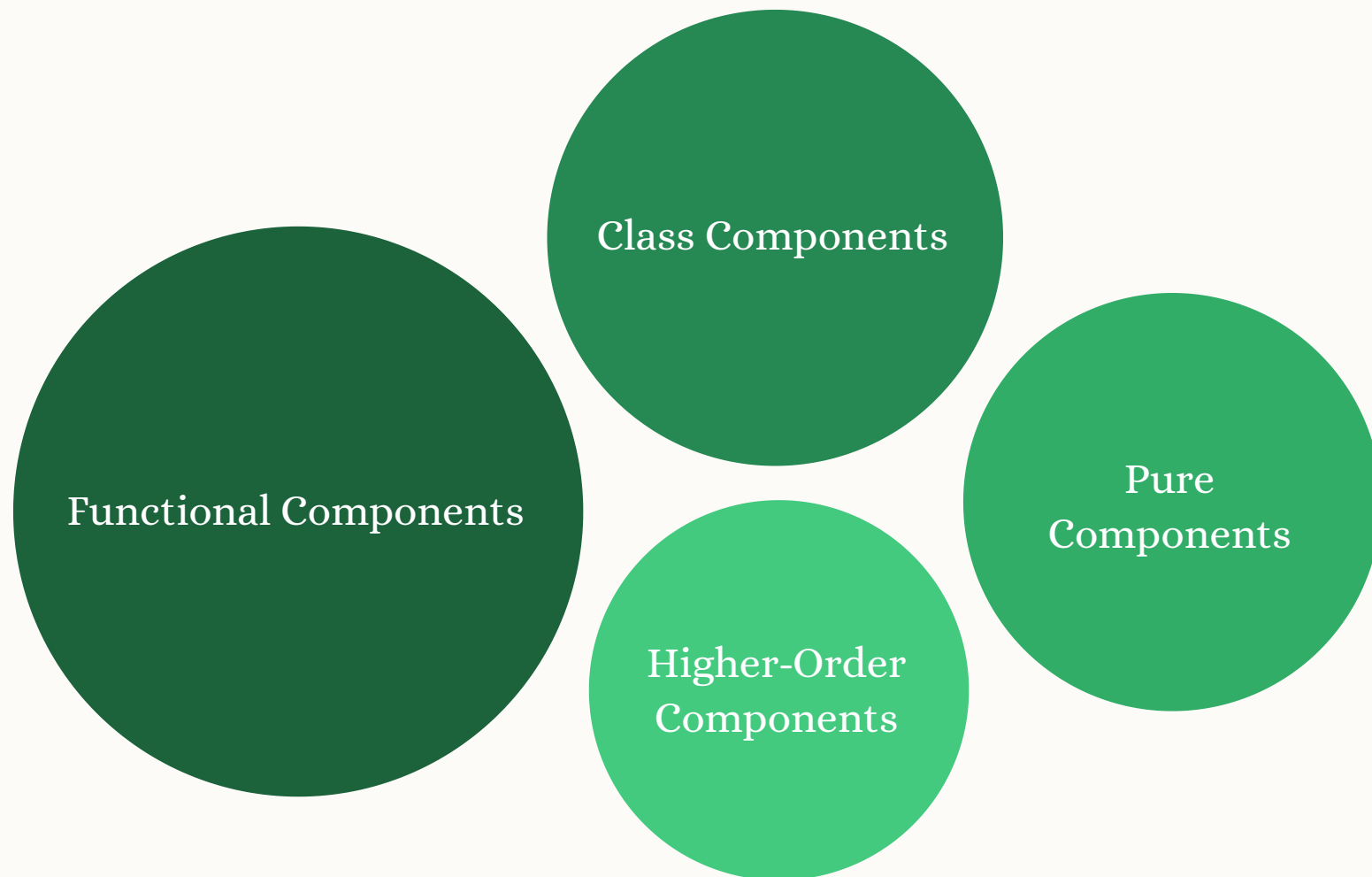
Why Components Matter

Components enable developers to break down complex user interfaces into smaller, manageable pieces. This modular approach offers several advantages:

- **Reusability:** Write once, use multiple times across the application
- **Maintainability:** Isolated code makes debugging and updates simpler
- **Scalability:** Applications grow without becoming unwieldy
- **Collaboration:** Teams can work on different components simultaneously



Types of Components in React



Functional Components

Modern, preferred approach using JavaScript functions. Introduced with React Hooks (16.8+), functional components can now handle state and lifecycle methods, making them the standard choice for new development.

```
function Welcome(props) {
  return <h1>Hello, {props.name}</h1>;
}

export default Welcome;
```

Key Features: Simpler syntax, easier to test, better performance, full access to React Hooks (useState, useEffect, etc.)

Class Components

Traditional approach using ES6 classes, extending React.Component. Whilst still supported, class components are now considered legacy code in favour of functional components with Hooks.

```
class Welcome extends React.Component {
  render() {
    return <h1>Hello, {this.props.name}</h1>;
  }
}

export default Welcome;
```

Key Features: Use 'this' keyword, render() method required, lifecycle methods (componentDidMount, etc.), verbose syntax

STRUCTURE

Anatomy of a React Component

01

Component Definition

Define the component as a function (or class). The function name should be capitalised, following React conventions. This function accepts props as a parameter.

02

JSX Return Statement

The component returns JSX (JavaScript XML), a syntax extension that resembles HTML. JSX describes the UI structure and must return a single parent element. Use fragments (`<>...</>`) to group multiple elements without adding extra DOM nodes.

03

Component Export

Export the component using either default or named exports. Default exports (export default ComponentName) allow importing with any name, whilst named exports require specific import syntax.

```
// Complete Component Example
import React from 'react';

function UserProfile(props) {
  return (
    <div className="profile">
      <h2>{props.name}</h2>
      <p>{props.email}</p>
    </div>
  );
}

export default UserProfile;
```

❏ **Important:** JSX expressions must be wrapped in curly braces `{}`. Component names must start with a capital letter to differentiate from HTML tags.



COMPOSITION

Component Reusability and Composition

The Power of Reusability

React components are designed to be reused throughout an application. A single component definition can be instantiated multiple times with different properties (props), generating varied output whilst maintaining consistent behaviour and structure.

```
// Reusing a Button component
<Button label="Submit" color="blue" />
<Button label="Cancel" color="grey" />
<Button label="Delete" color="red" />
```

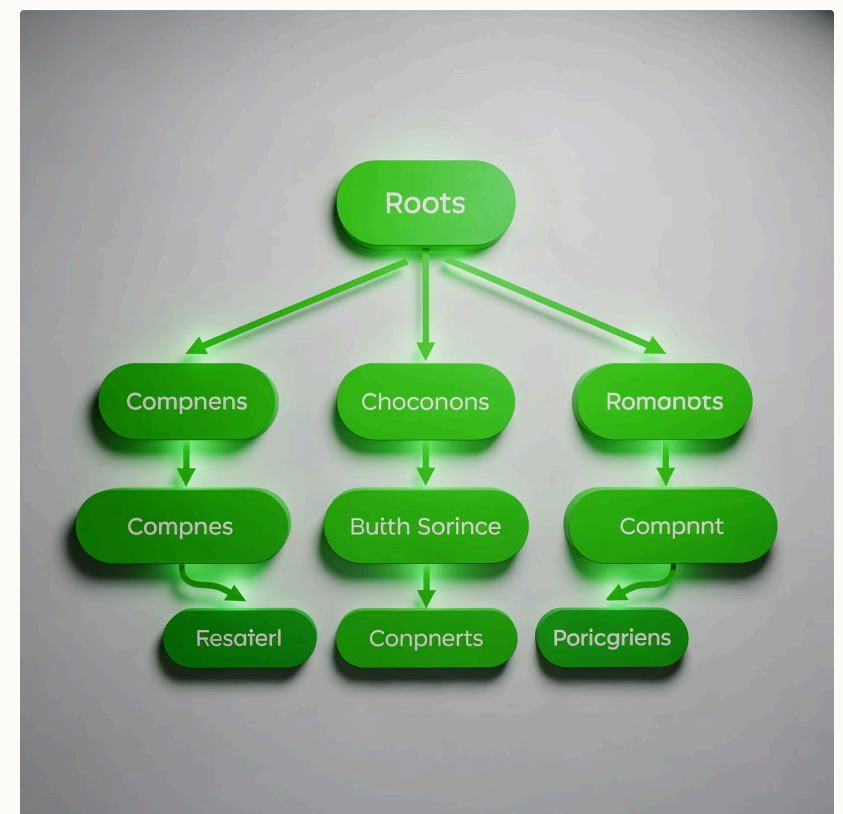
Component Composition

Composition involves nesting components within other components to build complex UIs from simple building blocks. Parent components contain child components, creating a hierarchical structure.

```
function App() {
  return (
    <div>
      <Header />
      <MainContent>
        <Sidebar />
        <Article />
      </MainContent>
      <Footer />
    </div>
  );
}
```

Benefits of Component-Based Design

- **Separation of Concerns:** Each component handles one specific part of the UI
- **Code Reusability:** Reduces duplication and development time
- **Easy Testing:** Components can be tested in isolation
- **Better Organisation:** Clear project structure and file organisation
- **Enhanced Collaboration:** Multiple developers can work independently on different components
- **Improved Performance:** React efficiently updates only changed components



Practical Component Example

1

Create the Component File

Create a new file named StudentCard.jsx in your components folder. Use .jsx extension to indicate the file contains JSX syntax.

2

Define Component Logic

```
function StudentCard({ name, rollNo,
branch, semester }) {
  const getGreeting = () => {
    return `Welcome, ${name}!`;
  };

  return (
    <div className="student-card">
      <h3>{getGreeting()}</h3>
      <p><strong>Roll No:</strong>
{rollNo}</p>
      <p><strong>Branch:</strong>
{branch}</p>
      <p><strong>Semester:</strong>
{semester}</p>
    </div>
  );
}
```

3

Use the Component

```
// In App.jsx or another parent
component
import StudentCard from
'./components/StudentCard';

function App() {
  return (
    <div>
      <StudentCard
        name="Rahul Kumar"
        rollNo="21CS001"
        branch="Computer Science"
        semester="2nd"
      />
    </div>
  );
}
```

This example demonstrates props destructuring, helper functions within components, and JSX interpolation. The component is self-contained and can be reused with different student data throughout the application.



SUMMARY

Key Takeaways: React Components



Core Concept

Components are independent, reusable code pieces that return UI elements. They serve as the building blocks of React applications, encapsulating structure, styling, and behaviour.



Component Types

Functional components (modern, Hook-based) and class components (legacy, lifecycle methods). Functional components are now the industry standard for new development.



Composition Model

Components nest within each other to create complex UIs from simple parts. This compositional approach enables scalability and maintainability in large applications.

Essential Points for Examinations

2-Mark Questions

- Define a React component
- Differentiate functional vs class components
- What is JSX?
- Why use component-based architecture?
- Explain props in components

5-Mark Questions

- Explain component lifecycle with examples
- Demonstrate component reusability with code
- Compare functional and class components in detail
- Describe component composition with practical example
- Explain advantages of component-based design in React

 **Remember:** Components promote reusability, maintainability, and scalability. Master the distinction between functional and class components, understand JSX syntax, and practise writing clean, modular component code for your examinations.